Zaawansowane techniki programowania

Laboratorium 10

ZMIENDAK WIKTOR

ZADANIA OBOWIĄZKOWE:

1. Zamiast pozwalać serwisowi na bezpośrednie publikowanie zdarzeń lub wywoływanie logiki magazynowej, zmodyfikowany OrderServiceImpl zapisuje utworzony event w komponencie OutboxStore.

```
@Override
public Order createOrder(long productId, int quantity) {
  if (quantity <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be > 0");
  }

  Instant now = Instant.now(clock);
  Order order = orderStore.create(productId, quantity, now);

  OrderCreatedEvent event = new OrderCreatedEvent(
      UUID.randomUUID().toString(),
      now,
      order.getId(),
      productId,
      quantity);

  outboxStore.save(routingKey: "orders.created", event, now);
  return order;
}
```

2. Przy tworzeniu zamówienia metoda createOrder najpierw waliduje dane, następnie tworzy obiekt Order w OrderStore, a potem buduje obiekt OrderCreatedEvent zawierający unikalne eventId.

```
@Override
public Order createOrder(long productId, int quantity) {
  if (quantity <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be > 0");
  }

  Instant now = Instant.now(clock);
  Order order = orderStore.create(productId, quantity, now);

  OrderCreatedEvent event = new OrderCreatedEvent(
      UUID.randomUUID().toString(),
      now,
      order.getId(),
      productId,
      quantity);

  outboxStore.save(routingKey: "orders.created", event, now);
  return order;
}
```

3. ProcessPending() pobiera listę nieprzetworzonych eventów z OutboxStore a następnie dla każdego eventu wywołuje messageBroker.publish(). Tylko po pomyślnym opublikowaniu eventu metoda oznacza go jako przetworzony, wywołując markProcessed().

```
public int processPending() {
  List<OutboxEvent> pending = outboxStore.findUnprocessed();
  int published = 0;

  for (OutboxEvent event : pending) {
    messageBroker.publish(event.getRoutingKey(), event.getPayload());
    event.markProcessed(Instant.now(clock));
    published++;
  }

  return published;
}
```

4. StockReservationSaga.handle() rozpoczyna od próby zamarkowania eventu jako obsłużonego w ProcessedEventsStore jeżeli tryMarkProcessed zwraca false, metoda natychmiast kończy działanie. Jeśli event jest nowy, wywoływane jest productCatalog.reserve(), aby zmniejszyć stan magazynowy.

```
public void handle(OrderCreatedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }

  try {
    productCatalog.reserve(event.productId(), event.quantity());
    StockReservedEvent reserved = new StockReservedEvent(
        UUID.randomUUID().toString(),
        Instant.now(clock),
        event.orderId(),
        event.productId(),
        event.quantity());
    messageBroker.publish(routingKey: "stock.reserved", reserved);
  } catch (ProductNotFoundException | InsufficientStockException e) {
    StockReservationFailedEvent failed = new StockReservationFailedEvent(
        UUID.randomUUID().toString(),
        Instant.now(clock),
        event.orderId(),
        event.productId(),
        event.quantity(),
        e.getMessage());
    messageBroker.publish(routingKey: "stock.reservation.failed", failed);
  }
}
```

5. Jeżeli podczas próby rezerwacji productCatalog.reserve() rzuci ProductNotFoundException albo InsufficientStockException, StockReservationSaga łapie wyjątek, tworzy StockReservationFailedEvent zawierający komunikat wyjątku.

6. OrderStatusListener obsługuje zarówno StockReservedEvent, jak i StockReservationFailedEvent. Jeżeli event jest nowy, w przypadku StockReservedEvent wywołuje orderStore.confirm(), co ustawia status zamówienia na CONFIRMED, natomiast w przypadku StockReservationFailedEvent wywołuje orderStore.reject(), co ustawia status na REJECTED i zapisuje powód odrzucenia.

```
public OrderStatusListener(OrderStore orderStore, ProcessedEventsStore processedEventsStore) {
  this.orderStore = orderStore;
  this.processedEventsStore = processedEventsStore;
}

public void handle(StockReservedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }
  orderStore.confirm(event.orderId());
}

public void handle(StockReservationFailedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }
  orderStore.reject(event.orderId(), event.reason());
}
```

ZADANIA DODATKOWE:

7. Wywołanie StockReservationSaga.handle() najpierw próbuje zamarkować identyfikator eventu jako przetworzony w ProcessedEventsStore, i jeżeli event był już wcześniej obsłużony, metoda kończy działanie natychmiast, dzięki czemu stan magazynowy nie jest zmniejszany.

```
public void handle(OrderCreatedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }

  try {
```

8. Każdy handler w OrderStatusListener zaczyna od próby oznaczenia eventId jako przetworzonego w ProcessedEventsStore; jeśli event był już obsłużony, handler zwraca bez zmiany stanu zamówienia.

```
public void handle(StockReservedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }
  orderStore.confirm(event.orderId());
}

public void handle(StockReservationFailedEvent event) {
  if (!processedEventsStore.tryMarkProcessed(event.eventId())) {
    return;
  }
  orderStore.reject(event.orderId(), event.reason());
}
```

9. Publikacja do MessageBroker wykonywana jest przed oznaczeniem eventu jako przetworzonego, a dopiero po pomyślnej publikacji wywoływane jest markProcessed(). W efekcie, jeśli publikacja zakończy się rzuceniem wyjątku, event pozostaje nieoznaczony jako przetworzony.

```
for (OutboxEvent event : pending) {
  messageBroker.publish(event.getRoutingKey(), event.getPayload());
  event.markProcessed(Instant.now(clock));
  published++;
}
```

10. i 11. Opisane wcześniej.

WYNIKI TESTÓW

```
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running pl.example.lab10.Grade3ArchitectureTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.101 s -- in pl.example.lab10.Grade3Architecture
[INFO] Running pl.example.lab10.Grade3OrderCreatesOutboxEventTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.150 s -- in pl.example.lab10.Grade3OrderCreates
oxEventTest
[INFO] Running pl.example.lab10.Grade3OrderStatusListenerTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in pl.example.lab10.Grade3OrderStatusL
nerTest
[INFO] Running pl.example.lab10.Grade3OutboxProcessorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in pl.example.lab10.Grade3OutboxProces
est
[INFO] Running pl.example.lab10.Grade3StockReservationSagaTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in pl.example.lab10.Grade3StockReserva
SagaTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.419 s
[INFO] Finished at: 2026-05-18T22:36:53+02:00
[INFO] ------------------------------------------------------------------------
```

```
: Initializing Spring TestDispatcherServlet ''
2026-05-18T22:37:26.414+02:00  INFO 5540 --- [lab10-outbox-saga] [           main] o.s.t.web.servlet.TestDispatcherServlet
: Initializing Servlet ''
2026-05-18T22:37:26.414+02:00  INFO 5540 --- [lab10-outbox-saga] [           main] o.s.t.web.servlet.TestDispatcherServlet
: Completed initialization in 0 ms
2026-05-18T22:37:26.422+02:00  INFO 5540 --- [lab10-outbox-saga] [           main] pl.example.lab10.Grade4ApiWorkflowTest
: Started Grade4ApiWorkflowTest in 0.472 seconds (process running for 6.245)
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.579 s -- in pl.example.lab10.Grade4ApiWorkflowTest
[INFO] Running pl.example.lab10.Grade4OrderStatusIdempotencyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in pl.example.lab10.Grade4OrderStatusIdemp
otencyTest
[INFO] Running pl.example.lab10.Grade4OutboxFailureTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in pl.example.lab10.Grade4OutboxFailureTes
t
[INFO] Running pl.example.lab10.Grade4SagaIdempotencyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in pl.example.lab10.Grade4SagaIdempotencyT
est
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  9.657 s
[INFO] Finished at: 2026-05-18T22:37:26+02:00
[INFO] ------------------------------------------------------------------------
```